CSE 4502

Operating System

ADVANCED SYNCHRONIZATION - FILE SHARING

Project 07

Team 04

OVERVIEW

- File-level synchronization in mCertiKOS
- Kernel implementation of flock-like system call
- Manages concurrent file access safely
- Demonstrates synchronization via user-level application

What is flock?

A Linux system call for managing file access in concurrent processes.

Types of Locks

Shared Lock
LOCK_SH

Multiple threads can read.

Exclusive Lock
LOCK_EX

Only one writer allowed, blocks others

Why flock is necessary?

- Prevents data corruption in multithreaded apps.
- Ensures safe concurrent file access.

Why flock?

Built-in Linux solution for reliable file locking.

Implementing Linux-style Locks in mCertiKOS

file_lock Structure

```
typedef struct flock_t
{
    spinlock_t lock;        // Protects the flock structure
    enum flock_state state; // Current lock state
    int shared_holders;     // Number of shared lock holders
    int waiting_shared;     // Number of waiting shared lock requests
    int waiting_exclusive;  // Number of waiting exclusive lock requests

    // Wait queue (per-inode): tracks processes waiting for this lock.
    // Blocking callers sleep on the channel equal to this flock_t pointer.
    struct
    {
        unsigned int pids[NUM_IDS];
        unsigned int len;
        uint8_t inq[NUM_IDS];
        uint8_t req[NUM_IDS]; // LOCK_SH or LOCK_EX, indexed by pid
    } waitq;
} flock_t;
```

Shared Lock (LOCK_SH)

```c
static int try_acquire_shared(flock_t *fl, int nonblocking)
{
    unsigned int pid = get_curid();

    if (fl->state == FLOCK_EXCLUSIVE || fl->waiting_exclusive > 0)
    {
        if (nonblocking)
        {
            return -1; // Would block
        }

        waitq_add(fl, pid, LOCK_SH);
        fl->waiting_shared++;
        while (fl->state == FLOCK_EXCLUSIVE || fl->waiting_exclusive > 0)
        {
            thread_sleep(fl, &fl->lock);
        }
        fl->waiting_shared--;
        waitq_remove(fl, pid);
    }

    fl->state = FLOCK_SHARED;
    fl->shared_holders++;
    return 0;
}
```

Exclusive Lock (LOCK_EX)

```c
static int try_acquire_exclusive(flock_t *fl, int nonblocking)
{
    unsigned int pid = get_curid();

    // Can acquire exclusive lock only if no locks are held
    if (fl->state != FLOCK_INACTIVE)
    {
        if (nonblocking)
        {
            return -1; // Would block
        }

        waitq_add(fl, pid, LOCK_EX);
        fl->waiting_exclusive++;
        while (fl->state != FLOCK_INACTIVE)
        {
            thread_sleep(fl, &fl->lock);
        }
        fl->waiting_exclusive--;
        waitq_remove(fl, pid);
    }

    fl->state = FLOCK_EXCLUSIVE;
    return 0;
}
```

Unlock (LOCK_UN)

```c
int flock_release(flock_t *fl, int lock_type)
{
    int need_wakeup = 0;

    spinlock_acquire(&fl->lock);

    switch (lock_type)
    {
    case LOCK_SH:
        if (fl->state == FLOCK_SHARED && fl->shared_holders > 0)
        {
            fl->shared_holders--;
            if (fl->shared_holders == 0)
            {
                fl->state = FLOCK_INACTIVE;
                need_wakeup = 1;
            }
        }
        break;

    case LOCK_EX:
        if (fl->state == FLOCK_EXCLUSIVE)
        {
            fl->state = FLOCK_INACTIVE;
            need_wakeup = 1;
        }
        break;
```

```c
    default:
        spinlock_release(&fl->lock);
        return -1;
    }

    if (need_wakeup)
    {
        thread_wakeup(fl);
    }

    spinlock_release(&fl->lock);
    return 0;
}
```

END-TO-END CALL PATH

When user code calls flock(fd, op), the flow is:

1. User call → flock(fd, op)
2. Syscall wrapper → sys_flock
3. Trap handler → dispatches system call
4. Kernel handler → processes request
5. File layer → manages file locking
6. Lock engine → acquire/release lock
7. Inode → stores lock state

TEST OVERVIEW

1. Basic Shared Lock
2. Basic Exclusive Lock
3. Multiple Shared Locks
4. Exclusive Blocks Shared
5. Shared Blocks exclusive
6. Lock Upgrade (SH → EX)
7. Lock Downgrade (EX → SH)
8. Auto-release on Close
9. Invalid FD
10. Write with Exclusive Lock
11. Read with Shared Lock
12. Multi-process Exclusive Blocks Child
13. Multi-process Shared Blocks Child Exclusive
14. Multi-process Child Succeeds After Release
15. Blocking Exclusive Waits For Shared Release
16. Blocking Shared Waits For Exclusive Release

DEMONSTRATION